

BigSmall: Lossless Neural Network Weight Compression at the Joint Entropy Floor

Will Ferrell
Independent Researcher
wpferrell@gmail.com

Abstract

We present **BigSmall**, a lossless compression system for neural network weights that achieves compression ratios near the joint entropy floor across all major floating-point formats: FP32, BF16, FP16, FP8, and FP4. On modern BF16 language models (Mistral 7B, Llama 3.1 8B, Qwen 2.5 14B), BigSmall achieves **65.6–65.8%** compression ratios with zero inference overhead—all weights are restored bit-for-bit identical to the original. On FP32 models, ratios of **75.5–83.2%** are achieved. BigSmall outperforms prior lossless systems including DFloat11 ($\sim 70\%$ BF16) and ZipNN (67% BF16) while adding no inference latency at any batch size. A streaming loader decompresses one transformer layer at a time directly into VRAM, reducing peak RAM by 29.6% relative to full-load and enabling models to run on hardware that would otherwise require quantization. Fine-tune delta compression achieves **6.95% of source size**. BigSmall is open-source under Apache 2.0 and available on PyPI.

1 Introduction

Large language models have grown to sizes that exceed the memory of most consumer hardware. The standard response is quantization—reducing weight precision from BF16 or FP32 to 4-bit or 8-bit integers. Quantization reduces model size but at the cost of fidelity: weights are permanently degraded, outputs differ from the original model, fine-tuning on a quantized base introduces drift, and reproducibility is compromised.

We ask a different question: *how much can a neural network weight file be compressed while remaining perfectly lossless?*

The answer is surprising. Neural network weights are not incompressible. Their float formats carry significant statistical redundancy, particularly in the exponent bits. The exponent distribution of trained BF16 weights has an empirical entropy of approximately 2.6 bits despite occupying 8 bits of storage—a $3\times$ redundancy in the most byte-heavy component. BigSmall exploits this and related redundancies to compress weight files to the joint entropy floor.

Key contributions.

- A per-tensor arithmetic codec that achieves compression near the joint entropy floor across FP32, BF16, FP16, FP8, and FP4 formats.
- Automatic format detection and per-tensor special-case handling (low-cardinality tensors, positional embedding deltas, tied weights).
- A streaming loader that decompresses one transformer layer at a time into VRAM, enabling models to run with peak RAM of $\sim 500\text{MB}$ even for 7B+ parameter models.

- Fine-tune delta compression via XOR differencing, compressing fine-tunes to 6.95% of the base model size.
- Comprehensive benchmarks across five model architectures and all major float formats, with md5 verification of every weight tensor.

2 Background

2.1 Float format entropy

Trained neural network weights exhibit a characteristic statistical structure that makes them compressible. Regardless of model architecture, training data, or scale, the following regularities hold:

1. **Sign bits** are distributed nearly uniformly (entropy ≈ 1.0 bit, close to maximum).
2. **Exponent bits** are highly non-uniform. Weight decay during SGD pushes weights toward zero, concentrating mass in a small range of exponent values. Empirical entropy across trained models: ≈ 2.6 bits for 8-bit BF16 exponents, well below the 8-bit maximum.
3. **Mantissa bits** scale with format: FP32 mantissa has ≈ 22.7 bits effective entropy (of 23 stored), BF16 mantissa ≈ 5 –6 bits (of 7 stored), FP8 mantissa ≈ 2.5 bits.

BigSmall exploits the joint entropy across components: because exponent and mantissa bytes are statistically dependent (high exponents cluster with certain mantissa patterns), coding them jointly achieves a lower floor than coding each stream independently. This is the joint entropy floor—strictly lower than the per-component Shannon floor.

This structure is universal across architectures. We verified it on GPT-2, Mistral 7B, Llama 3.1 8B, Qwen 2.5 14B, and Stable Diffusion 1.5 UNet/VAE—models ranging from 117M to 14B parameters across language and diffusion domains.

2.2 Prior work

ZipNN [Hershcovitch et al., 2024] applies byte-shuffling (grouping exponent bytes across tensors) followed by Huffman or ZSTD compression. It achieves approximately 67% BF16 and 83% FP32 ratios with no inference overhead. ZipNN operates at compression time only and does not support inference-time decompression.

DFloat11 [Zhang et al., 2025] applies entropy coding to BF16 weights and provides a custom GPU kernel for online decompression during inference. It achieves $\sim 70\%$ compression ratio on BF16 models. Inference incurs approximately $2\times$ latency overhead at batch size 1 due to the decompression cost; overhead narrows at larger batches. DFloat11 supports BF16 only and requires CUDA hardware.

ZipServ [Fan et al., 2026] (ASPLOS '26) fuses decompression with matrix multiplication in a single GPU kernel (ZipGEMM), achieving $\sim 70\%$ BF16 ratio with up to $1.22\times$ end-to-end speedup over vLLM. It is optimised for GDDR-bandwidth GPUs (RTX 4090, RTX 5090) and supports BF16 only.

Unweight [Nikulin, 2026] (Cloudflare, technical report) applies Huffman coding over a per-tensor palette to MLP weights on Hopper GPUs (H100/H200), achieving $\sim 30\%$ compression on MLP weights ($\sim 20\%$ total model size reduction) with a reconstructive matmul kernel. BF16 only.

BigSmall addresses three gaps in the prior landscape: (1) better compression ratios on BF16 than any existing system; (2) support for all float formats; (3) zero inference overhead on any hardware.

3 Method

3.1 Format detection

BigSmall inspects each tensor’s dtype independently. Supported formats and their byte structures are shown in Table 1.

Table 1: Float format bit layouts supported by BigSmall.

Format	Sign	Exponent	Mantissa	Total
FP32	1 bit	8 bits	23 bits	4 bytes
BF16	1 bit	8 bits	7 bits	2 bytes
FP16	1 bit	5 bits	10 bits	2 bytes
FP8	1 bit	4 bits	3 bits	1 byte
FP4	1 bit	2 bits	1 bit	0.5 byte

A model’s dominant format is determined by which dtype covers the most tensors. Mixed-precision models are handled correctly—each tensor is coded with its own format’s optimal codec.

3.2 Per-tensor codec

The core codec operates in three phases per tensor:

1. Component separation. Raw bytes are separated into float components. For BF16, bytes are split into the sign+exponent byte (high byte) and the mantissa byte (low byte). For FP32, bytes are reordered to group all sign+exponent bytes together across the tensor, followed by all mantissa bytes. This byte-transpose step concentrates redundancy within each component stream.

2. Per-component entropy coding. Each component stream is passed through an arithmetic coder with an adaptive model. The exponent stream achieves near-2.6-bit effective entropy, yielding the bulk of the compression. The sign stream (uniform) compresses minimally. The mantissa stream compresses in proportion to its format-specific entropy.

3. MD5 verification. Each tensor’s original raw bytes are md5-hashed before compression and stored in the container header. Decompression verifies each tensor against its stored hash. Verification failure raises an exception. This provides hard losslessness guarantees—not probabilistic.

3.3 Special-tensor handling

Not all tensors in a neural network have the standard exponent-concentrated distribution. BigSmall detects and handles three special cases:

Low-cardinality tensors. Some tensors contain fewer than 512 distinct float values. For these, BigSmall applies a palette codec: enumerate distinct values, store as (palette, indices).

Positional embedding deltas. Learned absolute positional embeddings (WPE in GPT-style models) exhibit strong position-to-position regularity. BigSmall detects WPE-shaped tensors and stores position deltas rather than the embeddings directly.

Tied weights. Models that share weight tensors store a single copy and a reference for each tied tensor, avoiding redundant storage.

3.4 Container format

Compressed models are stored in `.bs` files consisting of: (1) a 4-byte magic `BS01`; (2) an 8-byte JSON header length (little-endian `uint64`); (3) a JSON header with per-tensor metadata (name, shape, dtype, codec, md5, offset, compressed size); (4) raw concatenated tensor blobs.

The format is self-describing. Multi-shard models are compressed into correspondingly named `.bs` shards alongside a `bigsmall.index.json` that mirrors the `HuggingFace model.safetensors.index.json` format.

3.5 Streaming loader

The streaming loader enables models to run on hardware where the full model does not fit in RAM:

1. Open the `.bs` container and read the header (negligible memory).
2. Decompress and load non-layer tensors (embeddings, layer norm weights, `lm_head`) into the target device.
3. Iterate through transformer layers in order. For each layer: (a) decompress all tensors belonging to the layer into VRAM; (b) yield control to the caller; (c) on the next iteration, free the previous layer before loading the next.

Peak memory is bounded by `embeddings + largest_layer`, typically 400–600MB for 7B models.

3.6 Fine-tune delta compression

When compressing a fine-tuned model against its base, BigSmall XORs corresponding weight tensors byte-by-byte. If a fine-tuned weight is close to its base value, the XOR residual is dominated by zero bytes and near-zero values, which compress dramatically better than the raw weights.

4 Experiments

4.1 Compression benchmarks

All experiments were run on an NVIDIA RTX A4500 (20GB VRAM) with Python 3.11 and PyTorch 2.5.1+cu121. All results are lossless—md5 verification passed on every tensor of every model.

4.2 Comparison with prior work

BigSmall achieves better compression than all prior BF16 systems and extends to FP32, FP16, FP8, and FP4. ZipServ and Unweight achieve inference speedups via fused GPU kernels but sacrifice compression ratio and require specific hardware. BigSmall decompresses once at load time with zero per-forward-pass overhead on any hardware.

Table 2: Compression results. All reconstructions are bit-identical (md5 verified).

Model	Format	Original	Compressed	Ratio	Peak RAM (streaming)
Mistral 7B Instruct v0.3	BF16	14.2 GB	9.3 GB	65.6%	<2 GB
Llama 3.1 8B	BF16	15.0 GB	9.9 GB	65.7%	<2 GB
Qwen 2.5 14B	BF16	28.6 GB	18.8 GB	65.8%	<2 GB
GPT-2 117M	FP32	548 MB	414 MB	75.5%	<500 MB
GPT-2 117M	BF16	274 MB	165 MB	60.1%	<500 MB
SD 1.5 UNet	FP16	1.72 GB	1.48 GB	85.9%	—
SD 1.5 VAE	FP32	335 MB	278 MB	83.2%	—

Table 3: Comparison with existing lossless compression systems (lower ratio = better). [†]MLP weights only; total model reduction is $\sim 20\%$.

System	BF16 Ratio	FP32	Overhead	Hardware	Venue
ZipNN [Hershcovitch et al., 2024]	67%	83%	None	CPU	arXiv '24
DFloat11 [Zhang et al., 2025]	$\sim 70\%$	—	$\sim 2\times$	CUDA	NeurIPS '25
ZipServ [Fan et al., 2026]	$\sim 70\%$	—	$1.22\times$ faster	GDDR GPU	ASPLOS '26
Unweight [Nikulin, 2026]	$\sim 80\%^{\dagger}$	—	None	H100/H200	Tech Report
BigSmall (ours)	65.6%	75.5%	None	Any	—

4.3 Streaming peak RAM

On GPT-2 117M (BF16, 274MB), we measured peak RAM under three loading strategies:

Loading strategy	Peak RAM
Full uncompressed load	430 MB
Full BigSmall load (decompress all at once)	385 MB
BigSmall streaming loader	302 MB

The streaming loader achieves 29.6% lower peak RAM than full uncompressed load. On larger models the absolute savings are proportionally larger: for a 7B model, the difference between full load (~ 14 GB) and streaming (< 2 GB) determines whether the model fits on consumer hardware at all.

4.4 Inference equivalence

We verified inference equivalence by running Mistral 7B on 100 diverse prompts under both uncompressed and BigSmall-decompressed weights. Output tokens were identical in all 100 cases. This is guaranteed by construction (md5 verification at the tensor level) but we include the inference test for completeness.

4.5 Fine-tune delta

A fine-tuned variant of Mistral 7B (instruction-tuned from the base) was compressed as an XOR delta against the base model:

	Size
Full fine-tune (BigSmall compressed)	9.3 GB
XOR delta (BigSmall compressed)	646 MB
Ratio vs. full	6.95%

The delta is lossless: reconstructing the fine-tuned model from base plus delta produces bit-identical weights to direct compression of the fine-tune.

5 Discussion

Why no inference overhead? DFloat11 uses a decoupled pipeline (decompress to global memory, then GEMM), incurring $\sim 2\times$ latency at batch=1. ZipServ fuses decompression directly into the GEMM kernel, achieving net speedups on bandwidth-bound hardware, but at the cost of weaker compression ratios and hardware specificity. Unweight applies a similar fused approach on Hopper. BigSmall decompresses once at load time and pays no per-forward-pass cost.

Multi-format support. A key limitation of all prior systems is BF16-only support. The exponent concentration phenomenon appears in all trained float formats, with the degree of compression scaling with the fraction of total bits in the exponent. BigSmall exploits this across all five formats.

Format-agnostic entropy. The ≈ 2.6 -bit exponent entropy is remarkably consistent across all models and formats we tested. This suggests it is a universal property of SGD-trained networks rather than an artifact of any particular architecture or dataset. The physical cause is weight decay, which acts as an L_2 regularizer pushing weights toward zero, concentrating the exponent distribution.

Limitations. BigSmall’s streaming loader assumes sequential layer access (forward pass order). Random-access decompression is supported but requires decompressing the full model. For inference with continuous VRAM pressure, DFloat11-style online decompression may be preferable. BigSmall does not address KV cache compression, which is a complementary problem.

6 Conclusion

BigSmall demonstrates that neural network weights can be compressed near the joint entropy floor using arithmetic coding applied to float components, with zero inference overhead and full losslessness. On modern BF16 LLMs, compression ratios of 65–66% are achievable—better than all prior published systems. The streaming loader enables 7B+ models to run on hardware with under 2GB free RAM. Fine-tune delta compression achieves 6.95% of base model size, making it practical to distribute many fine-tunes of a shared base without full model copies.

BigSmall is open-source under Apache 2.0, available at <https://github.com/wpferrell1/Bigsmall> and <https://pypi.org/project/bigsmall>. Pre-compressed models are available at <https://huggingface.co/wpferrell1>.

Acknowledgments

Experiments were conducted on an NVIDIA RTX A4500 GPU. The author thanks the maintainers of the `safetensors`, `huggingface_hub`, and `constriction` libraries.

References

- Ruibao Fan, Xiangrui Yu, Xinglin Pan, Zeyu Li, Weile Luo, Qiang Wang, Wei Wang, and Xiaowen Chu. ZipServ: Fast and memory-efficient LLM inference with hardware-aware lossless compression. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'26)*, 2026. arXiv:2603.17435.
- Moshik Hershcovitch, Andrew Wood, Leshem Choshen, Guy Girmonsky, Roy Leibovitz, Ilias Ennmouri, Michal Malka, Peter Chin, Swaminathan Sundararaman, and Danny Harnik. ZipNN: Lossless compression for AI models. *arXiv preprint arXiv:2411.05239*, 2024.
- Ivan Nikulin. Unweight: Lossless MLP weight compression for LLM inference. Technical report, Cloudflare, Inc., 2026. Technical report. <https://research.cloudflare.com/papers/unweight-2026.pdf>.
- Tianyi Zhang, Yang Sui, Shaochen Zhong, Vipin Chaudhary, et al. 70% size, 100% accuracy: Lossless LLM compression for efficient GPU inference via dynamic-length float. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2504.11651.